

05 · 핵심 코드 해설 — 백엔드 RAG와 DB의 연결

제안서의 "기술적 실현 가능성·수행 능력"

0. 한 장 요약 — 데이터 흐름

```
[수집]  제품명 → 리뷰 수집 → 청킹 → 임베딩 → chunks 테이블 (벡터 컬럼)
                                             ↳ LLM 라벨링 (카테고리·감성·여정) → 5축 평점
[질의]  질문 → 질의 임베딩 → 벡터 유사도 검색 (전문가/일반 분리 top-k, 전문가 우선)
                                             → LLM이 근거 인용 답변 생성 (근거 밖 내용 금지)
```

전환 전(M6)에는 "질의 임베딩"과 "벡터 검색"이 **DB 밖** 서비스였고, 전환 후(Oracle)에는 둘 다 **DB 안 SQL**로 들어갔습니다. 이 차이가 이 프로젝트의 핵심 서사입니다.

1. RAG 질의 경로 — 하이브리드 검색의 설계 의도

`ai-worker/src/rag/answer.py` (파이프라인 원형, Python):

```
qv = embed_one(query)
# ★ 하이브리드 검색: 전문가/일반을 한 번에 top-k 하지 않고 "따로" 뽑아 합침.
#   이렇게 안 하면 한쪽(보통 양 많은 일반 후기)이 상위를 독식 → 전문가 근거가 밀림.
retrieved = []
if expert_k > 0:
    retrieved += _retrieve(qv, ..., source_type="expert", k=expert_k)
if public_k > 0:
    retrieved += _retrieve(qv, ..., source_type="public_review", k=public_k)
```

해설: DILAB 검색의 차별점이 이 주석에 그대로 있습니다. 일반 리뷰가 수적으로 우세하므로 단일 top-k로는 전문가 근거가 밀립니다. **출처 유형별로 따로 뽑아 전문가를 우선 결합**하는 정책이 "전문가급 신뢰성"의 구조적 장치입니다. 이후 LLM 합성 단계는 시스템 프롬프트로 **근거 밖 내용 금지·모든 주장에 인용 번호 부착**을 강제합니다 (`prototype/lib/rag.ts`의 `SYSTEM_PROMPT`).

2. 벡터 검색 — 전환 전/후 대비 (가장 중요한 발체)

2-A. 전환 전 (M6): 외부 임베딩 + RPC 왕복

`prototype/lib/embeddings.ts` + `prototype/lib/rag.ts` (M6 시점, git 이력 `afb7866`):

```
// 질의 임베딩 - 서버리스 AI 바인딩 (BGE-M3, 1024차원)
const r = await env.AI.run("@cf/baai/bge-m3", { text });

// 벡터 검색 - Supabase RPC를 source_type별로 2회 호출
const { data } = await supabase.rpc("match_chunks", {
  query_embedding: qv,          // 1024개 실수를 네트워크로 전송
  match_domain_id: domainId,
  match_source_type: sourceType,
  match_count: k,
});
```

match_chunks 의 몸통 SQL (docs/db/schema.md):

```
select c.id, c.text, d.source_type, d.author,
       1 - (c.embedding <=> $1::vector) as similarity
from chunks c join documents d on d.id = c.document_id
where c.domain_id = $2 and d.source_type in ('expert','public_review')
order by case d.source_type when 'expert' then 0 else 1 end, -- 전문가 우선
          c.embedding <=> $1::vector                        -- 그다음 유사도
limit 10;
-- 인덱스: HNSW (m=16, ef_construction=64), 컬럼 vector(1024)
```

해설: 질의 1건당 **외부 임베딩 API 1회 + 벡터 배열 전송 + RPC 2회**의 네트워크 왕복이 발생합니다. 검색 자체는 코사인 유사도(`<=>`) + 관계형 필터(도메인·출처) + 전문가 우선 정렬이며, 근사 인덱스(HNSW)를 씁니다.

2-B. 전환 후 (Oracle 26ai): 임베딩+검색이 SQL 1회

prototype/lib/rag.ts (현재, 핵심):

```
WITH q AS (SELECT VECTOR_EMBEDDING(DILAB_E5 USING 'query: ' || :qtext AS data) AS qv
           FROM dual)                                -- ① 질의 임베딩을 DB 안에서 1회

SELECT chunk_id, text, source_type, author, similarity
FROM (
  SELECT c.id AS chunk_id, c.text, c.source_type, c.author,
         (1 - VECTOR_DISTANCE(c.embedding, q.qv, COSINE)) AS similarity,
         ROW_NUMBER() OVER (PARTITION BY c.source_type                -- ② 출처별 top-k
                           ORDER BY VECTOR_DISTANCE(c.embedding, q.qv, COSINE)) AS rn
  FROM chunks c CROSS JOIN q
  WHERE c.domain_id = :domain AND c.source_type IN ('expert','public_review')
)
WHERE (source_type = 'expert' AND rn <= :ek) OR (source_type = 'public_review' AND rn <= :pk)
ORDER BY CASE source_type WHEN 'expert' THEN 0 ELSE 1 END, similarity DESC -- ③ 전문가 우선
```

해설: 전환 전의 세 가지 왕복(외부 임베딩 1 + RPC 2)이 **SQL 한 번**으로 접혔습니다. `VECTOR_EMBEDDING(DILAB_E5 ...)` 이 DB에 적재된 ONNX 모델로 질의를 임베딩하고, 같은 문장 안에서 `VECTOR_DISTANCE` 로 검색까지 끝냅니다.

`ROW_NUMBER() OVER (PARTITION BY source_type)` 가 pgvector 시절의 "출처별 분리 top-k + 전문가 우선" 정책을 1:1로 재현합니다. 'query: ' 접두어는 e5 계열 모델의 비대칭 검색 규약입니다. **질의·데이터가 DB 경계를 나가지 않는다** — 거버넌스 서사의 코드 증거입니다.

2-C. 동등성 검증

`scripts/validate_match_chunks_full.py` : pgvector `match_chunks` 의 인자(출처 필터, 전문가 우선)를 Oracle SQL로 재현해 **Top-k 순서 완전일치**를 확인 — "이관 후 검색 결과가 달라지지 않았다"의 직접 근거 (docs/ORACLE_MIGRATION_PLAN.md 에 결과 기록).

3. 분석 파이프라인 — 서버리스 워커와 DB의 왕복

`modal_app/analyze.py` (서버리스 GPU 워커):

```
@app.function(cpu=1, memory=8192, timeout=900, ...)
def analyze_product_task(job_id, product_query, domain_slug):
    """무거운 분석 - 웹 API의 trigger가 .spawn()으로 호출(fire-and-forget).
    진행 상태는 DB의 analysis_jobs 테이블로 왕복, 클라이언트는 polling."""
    update(status="running", progress={"step": 1, "of_steps": 3, ...})
    result = analyze_product(product_query, domain_slug)  # ← ai-worker 코드 재사용
    update(status="done", result_slug=result["slug"], ...)
```

`ai-worker/src/ingestion/auto_ingest.py` — `analyze_product()` 의 5단계:

```
① 리뷰 수집 → ② 도메인/제품 upsert → ③ documents.chunks 적재(+임베딩)
→ ④ LLM 라벨링 label_domain() - 청크당 1회 호출로 카테고리+감성+구매여정 동시 판정
→ ⑤ 5축 평점 upsert_ratings() - axis_score = ((mean(가중 감성) + 1) / 2) * 10,
                                     축별 근거 청크 5개(evidence_chunk_ids) 자동 선별
```

해설: 무거운 분석은 서버리스 컨테이너에서 돌고, **작업 큐·진행률·결과가 전부 DB 테이블(analysis_jobs)로 오갑니다** — 별도 큐 인프라가 없는 단순한 구조. 라벨링 결과는 `classifications / sentiments / journey_assignments` 3테이블에, 평점은 `ratings` 테이블에 근거 청크 id와 함께 저장되어 **화면의 모든 점수를 원문 근거로 역추적**할 수 있습니다. 초기의 로컬 FastAPI 워커(ai-worker)는 폐기된 것이 아니라 이 서버리스 컨테이너로 **코드 그대로 이식**되었습니다.

4. Oracle 실험·적재 코드 — 논문 3조건의 구현

4-A. 실험 조건 B/C의 검색 SQL (scripts/paper_experiment.py)

```
-- 조건 B: A와 "동일한" 1024d 벡터를 바인딩, 저장소만 Oracle (정확검색)
SELECT id, text FROM chunks
WHERE domain_id = :d AND embedding_bge IS NOT NULL
ORDER BY VECTOR_DISTANCE(embedding_bge, :q, COSINE)
FETCH FIRST :k ROWS ONLY

-- 조건 C: 질의 텍스트만 넘기면 임베딩+검색이 DB 안 단일 SQL
WITH q AS (SELECT VECTOR_EMBEDDING(DILAB_E5 USING 'query: ' || :t AS data) v FROM dual)
SELECT c.id, c.text FROM chunks c, q
WHERE c.domain_id = :d AND c.embedding IS NOT NULL
ORDER BY VECTOR_DISTANCE(c.embedding, q.v, COSINE)
FETCH FIRST :k ROWS ONLY
```

해설: 조건 B는 원본과 **같은 벡터**를 실어 "저장소 요인"만 남기고, 조건 C는 임베딩 위치까지 DB 안으로 넣어 "위치 요인"을 엮었습니다. 논문의 브리지 조건이 코드 수준에서 어떻게 공정 비교를 만드는지 보여 주는 대목입니다.

4-B. ONNX 모델 적재 — 외부 임베딩 서버의 제거 (scripts/load_onnx_model.py)

```
cur.execute(
    "BEGIN DBMS_VECTOR.LOAD_ONNX_MODEL("
    "model_name => :name, model_data => :blob, metadata => JSON(:meta)); END;",
    name="DILAB_E5", blob=onnx_blob, ...)
# 검증: SELECT VECTOR_EMBEDDING(DILAB_E5 USING '수분크림 진정 효과' AS data) FROM dual
# → "임베딩 생성됨 - 차원 384 (외부 임베더 없이 DB가 직접)"
```

4-C. 코퍼스 전체 재임베딩 — UPDATE 한 줄 (scripts/reembed_indb.py)

```
ALTER TABLE chunks ADD embedding VECTOR(384, FLOAT32);
UPDATE chunks SET embedding =
    VECTOR_EMBEDDING(DILAB_E5 USING 'passage: ' || text AS data); -- 1,347청크 일괄
CREATE VECTOR INDEX idx_chunks_emb ON chunks (embedding)
    ORGANIZATION INMEMORY NEIGHBOR GRAPH DISTANCE COSINE WITH TARGET ACCURACY 95;
```

해설: 코퍼스 재임베딩이 애플리케이션 루프 없이 **UPDATE 문 하나로** 끝납니다(문서 측 'passage: ', 질의 측 'query: ' — e5 비대칭 규약). 마지막 줄이 Oracle의 HNSW 대응 벡터 인덱스입니다.

4-D. 통계 검증 (scripts/paper_stats.py)

원자료(results_raw.json)에 대해 대응표본 t검정 · 윌콕슨 · 부트스트랩 CI(10,000회, seed=42) · Cohen's d_z · TOST 등가성(±0.15)을 일괄 산출 — 논문의 모든 수치가 이 스크립트로 재현됩니다.

5. 제안서에서 쓸 만한 문장으로

- "질의 임베딩과 하이브리드 벡터 검색을 **단일 SQL로 처리하는 in-database RAG 경로를 구현·검증**하였다 (외부 임베딩 API 왕복 제거)."
- "이전 전후의 검색 동등성을 **상위 결과 순서 완전일치** 수준에서 검증하는 절차와 도구를 보유하고 있다."
- "분석 파이프라인은 서버리스 워커와 DB 테이블 기반 작업 큐로 구성되어 **상시 서버 없이** 운영된다."
- "평가 점수는 근거 청크 식별자와 함께 저장되어 **모든 판단이 원문으로 역추적**된다."